

Disciplina: Aprendizado Profundo

Aula 4: Laboratório - Construindo um MLP com NumPy

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação, Universidade Federal do
Ceará (MDCC / UFC)

17 de Setembro de 2025

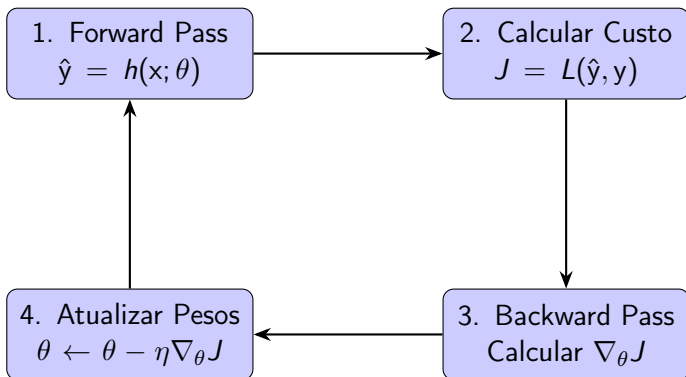
Roteiro da Aula de Hoje

- 1 Revisão para o Laboratório
- 2 Proposta do Laboratório: Um Mini-Framework
- 3 Passo a Passo da Implementação
- 4 Exercícios do Laboratório
- 5 Próximos Passos

O Ciclo de Treinamento de um MLP

Nosso Objetivo de Hoje

Implementar este ciclo de treinamento do zero, usando apenas NumPy.



O Objetivo: Construir um MLP Modular

Por que “do Zero”?

- Para entender profundamente a mecânica do backpropagation e da otimização.
- Para apreciar o que frameworks como PyTorch fazem por nós.
- Para construir uma base sólida que facilitará o aprendizado de arquiteturas mais complexas.

Problema 1

Vamos treinar uma rede para classificar o dataset *Duas Luas* (*Two Moons*), um problema clássico de classificação não-linear.

A Arquitetura do Nosso Código I

Para tornar nosso código flexível, vamos encapsular cada componente em sua própria classe.

- **Differentiable**: uma classe (interface) para todas as classes que implementam `forward` e `backward` e calculam gradientes. Por padrão o método `parameters()` dessa classe base retorna uma lista vazia.
- **Layer** : Uma classe base para a rede neural com métodos `forward(valor_entrada)` e `backward(gradiente_saida)`, além do método `parameters()` que retorna uma lista não vazia $[(\theta, d\theta) : \theta, d\theta \in \text{Layer}()]$ com pares de valores referentes aos parâmetros da camada e seus respectivos gradientes.

A Arquitetura do Nosso Código II

- Linear, Sigmoid, ReLU, etc.: Camadas concretas que herdam de Layer e Differentiable e implementam a lógica de seus cálculos forward e backward.
- Loss: Uma classe para a função de custo (e.g., MSELoss ou BinaryCE).
- Regularizer: Uma classe para a penalidade de regularização (e.g., L2Regularizer).
- Optimizer: Uma classe para o otimizador (e.g., SGD) que lida com a atualização dos pesos.
- Sequential: Uma classe que agrupa as camadas para formar o modelo final.

Passo 1: Camadas e Custo I

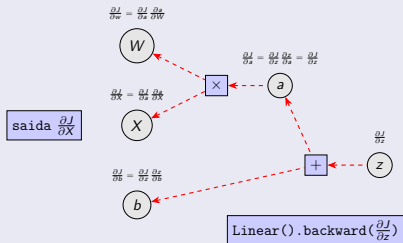
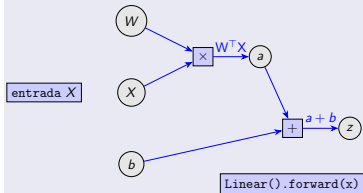
A Classe Linear

Implementa a transformação $z = W^T X + b$.

- forward: Recebe X , armazena para o backward pass, e retorna z .
- backward: Recebe $\frac{\partial J}{\partial z}$, calcula $\frac{\partial J}{\partial W}$, $\frac{\partial J}{\partial b}$ e $\frac{\partial J}{\partial X}$, armazena os gradientes dos parâmetros e retorna o gradiente em relação à entrada $\frac{\partial J}{\partial X}$.

Passo 1: Camadas e Custo II

Grafo Computacional do Linear



Passo 1: Camadas e Custo III

A Classe MSELoss

- forward: Recebe $\hat{y}$ e y e retorna o valor escalar do custo.
- backward: Não recebe nada e retorna o gradiente inicial do custo, $\frac{\partial J}{\partial \hat{y}} = \frac{2}{N}(\hat{y} - y)$.

Passo 2: O Otimizador

A Classe SGD

Responsável por aplicar a regra de atualização.

- `__init__(model, learning_rate)`: Recebe o modelo para ter acesso aos seus parâmetros.
- `zero_grad()`: Itera sobre as camadas do modelo e zera os gradientes armazenados (`.parameters()`).
- `step()`: Itera sobre as camadas do modelo e aplica a atualização de pesos:

$$W \leftarrow W - \eta \cdot dW$$

Passo 2.1: O Regularizador (Extra)

A Classe L2Regularizer

Calcula a penalidade $J_{reg} = \frac{\lambda}{2} \sum ||W||_F^2$.

- **Exercício teóricos:** mostre que esse regularizador é equivalente a *weight decay* dado por
$$W_{t+1} = (1 - \eta\lambda)W_t - \nabla_W J$$
- `forward(model)`: Itera sobre as camadas do modelo, calcula e retorna a penalidade L2 total.
- `backward(model)`: Itera sobre as camadas e **adiciona** o gradiente da regularização ao gradiente do peso já calculado, `layer.parameters()`.

$$dW = \nabla_W J + \nabla_W J_{reg}$$

Passo 3: O Loop de Treinamento Modular

```
# Para cada época:
# Para cada mini-batch:
# 1. Forward pass
y_pred = model.forward(X_batch)

# 2. Calcular o custo
data_loss = loss_fn.forward(y_pred, y_batch)
reg_loss = regularizer.forward(model)
total_loss = data_loss + reg_loss

# 3. Zerar gradientes anteriores
optimizer.zero_grad()

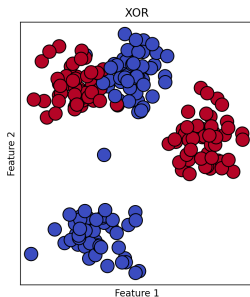
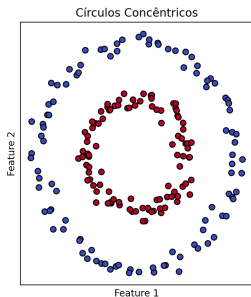
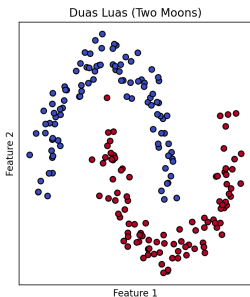
# 4. Backward pass
grad = loss_fn.backward()
model.backward(grad)
regularizer.backward(model)

# 5. Atualizar os pesos
optimizer.step()
```

Exercício 1: Carregar e Visualizar os Dados I

Conhecendo os Desafios

O primeiro passo de qualquer projeto de Machine Learning é entender os dados. Vamos trabalhar com datasets 2D clássicos que nos permitem visualizar facilmente as fronteiras de decisão.



Exercício 1: Carregar e Visualizar os Dados II

Figura: Datasets para classificação binária: Duas Luas, Círculos Concêntricos e XOR, MNIST. Todos são impossíveis de separar com um único modelo linear.

Tarefa

No notebook, carregue e plote cada um desses datasets usando as funções do `scikit-learn`.

Exercício 2: Construir e Treinar o MLP I

A Tarefa Principal

O coração do laboratório é implementar as classes modulares que projetamos (Linear, Sigmoid, MSELoss, SGD, etc.) e usá-las para treinar um MLP no dataset *Duas Luas*.

Exercício 2: Construir e Treinar o MLP II

Arquitetura do Modelo

Vamos construir um modelo `Sequential` com a seguinte arquitetura:

- Camada Linear (2 entradas, 16 saídas)
- Ativação Sigmóide
- Camada Linear (16 entradas, 1 saída)
- Ativação Sigmóide (para mapear a saída para uma probabilidade)

O objetivo é preencher os métodos `forward` e `backward` de cada componente e o método `step` do otimizador. A depender do tempo, implemente também variações da rede utilizando `ReLU` e regularização.

Exercício 3: Visualizar o Aprendizado I

Fronteira de Decisão

Após o treinamento, a tarefa é implementar uma função para visualizar a fronteira de decisão aprendida pelo seu MLP. Isso nos mostra como a rede define o espaço de features e separa as classes.

O que esperamos ver?

Uma fronteira não-linear que se ajusta à forma dos datasets.

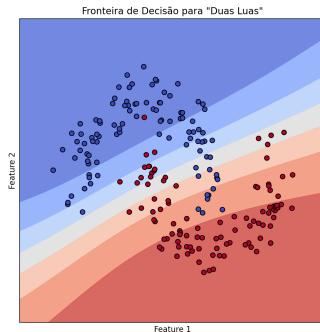


Figura: Exemplo de fronteira de decisão para o problema das duas luas.

Exercício 4: Inspeccionando a Rede I

Visualizando a Estrutura da Rede

Utilize o pacote `matplotlib` para gerar uma visualização da rede. Podem ser criativos aqui, mas uma sugestão é representar cada camada em um único bloco / retângulo, com texto representando o tamanho dos parâmetros e tipo de ativação.

Faça também um versão em apenas texto, adicionando o método `__repr__` ou `__str__` a classe.

Visualizando a Função de Ativação

Para garantir que sua implementação está correta, é útil plotar as funções de ativação. Crie uma pequena função para plotar a saída da sua camada Sigmoid para um intervalo de entradas, e verifique se o gráfico corresponde à forma esperada.

Exercício 5: Testando outros datasets e variações na arquitetura I

A Tarefa

Modifique sua rede e parâmetros de treinamento para resolver os datasets de círculos concêntricos, XOR e MNIST com dois dígitos apenas.

Exercício 6: Adaptando a Rede para Regressão I

A Tarefa

Modifique sua rede e loop de treinamento para resolver um problema de regressão: ajustar uma curva a dados ruidosos.

Exercício 6: Adaptando a Rede para Regressão II

Mudanças Necessárias

- 1 Dados:** Gere dados a partir de uma função (e.g., $y = 6 \sin(x) + 8 \cos(x) + x^2$) com ruído Gaussiano. Se possível cria a função também de forma aleatória escolhendo função simples e termos multiplicativos aleatórios.
- 2 Arquitetura:** A camada de saída **não deve** ter uma ativação Sigmóide, pois a saída não é uma probabilidade. Use uma camada Linear diretamente. As camadas internas podem ser ReLU ou Sigmoid.
- 3 Custo:** A função de custo deve ser MSELoss, que é a escolha padrão para regressão.

Exercício 6: Adaptando a Rede para Regressão III

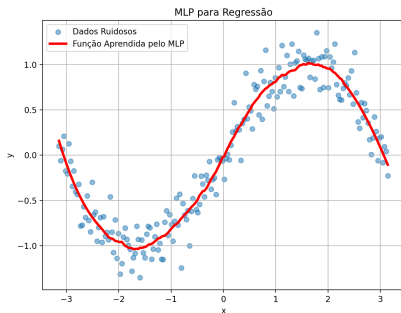


Figura: Exemplo de um MLP ajustando uma função seno.

Exercício 7: Compare sua implementação com uma implementação em PyTorch ou Jax I

A Tarefa

Escolha JAX ou PyTorch e desenvolva uma implementação padrão num desses para os problema de classificação tratados hoje. Compare os resultados obtidos.

Encerramento e Próximos Passos I

Resumo do Laboratório

- Implementamos os componentes de um MLP (camadas, ativações, custo) em classes.
- Construímos um otimizador e um regularizador modulares.
- Treinamos um MLP do zero em um problema não-linear, entendendo na prática cada passo do processo.

Encerramento e Próximos Passos II

Para a Próxima Aula

- **Tópico:** Funções de Custo, Regularização e Otimizadores.
- **Objetivo:** Vamos aprofundar a teoria por trás das escolhas que fizemos hoje, explorando diferentes funções de custo, técnicas de regularização e otimizadores mais avançados que o SGD.

Lista de Exercícios 1

A lista de exercícios, baseada neste laboratório, será disponibilizada no SIGAA. Todas as tarefas desenvolvidas em laboratório farão parte da lista, mas teremos mais exercícios.

Referências I

-  Bach, Francis (2025). *Learning Theory from First Principles*.
URL: https://www.di.ens.fr/~fbach/ltfp_book.pdf.
-  Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Bishop, Christopher M. e Hugh Bishop (2023). *Deep Learning: Foundations and Concepts*. Springer.
-  Buchanan, Sam et al. (ago. de 2025). *Learning Deep Representations of Data Distributions*.
<https://ma-lab-berkeley.github.io/deep-representation-learning-book/>. Online.
-  Cybenko, George (1989). "Approximation by superpositions of a sigmoidal function". Em: *Mathematics of control, signals and systems* 2.4, pp. 303–314.

Referências II

-  Domke, Justin (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf.
-  Gefer, Amanda (fev. de 2015). “The Man Who Tried to Redeem the World with Logic”. Em: *Nautilus*. URL: <https://nautil.us/the-man-who-tried-to-redeem-the-world-with-logic-235253/>.
-  Grünwald, Peter D. (2007). *The Minimum Description Length Principle*. The MIT Press.
-  Jordan, Michael I (2019). “Artificial intelligence—the revolution hasn’t happened yet”. Em: *Harvard Data Science Review* 1.1.
-  Krause, Andreas e Jonas Hübner (2025). *Probabilistic Artificial Intelligence*. arXiv: 2502.05244 [cs.AI].

Referências III

-  Liu, Yuxi (jan. de 2024). *The Perceptron Controversy*. Website. URL: <https://yuxi.ml/essays/posts/perceptron-controversy/> (acesso em 10/09/2025).
-  Martins, André FT e Ramón F Astudillo (2016). “From softmax to sparsemax: A sparse model of attention and multi-label classification”. Em: *International conference on machine learning*. PMLR, pp. 1614–1623.
-  McCulloch, Warren S. e Walter Pitts (1943). “A Logical Calculus of the Ideas Immanent in Nervous Activity”. Em: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133. DOI: [10.1007/bf02478259](https://doi.org/10.1007/bf02478259).

Referências IV

-  Mercer, James (1909). “Functions of positive and negative type, and their connection with the theory of integral equations”. *Em: Philosophical transactions of the Royal Society of London. Series A* 209.441-458, pp. 415–446.
-  Minsky, Marvin e Seymour A Papert (1969). *Perceptrons: An introduction to computational geometry*. MIT press.
-  Montufar, Guido F et al. (2014). “On the number of linear regions of deep neural networks”. *Em: Advances in neural information processing systems* 27.
-  Murphy, Kevin Patrick (2023). *Probabilistic Machine Learning: Advanced Topics*. The MIT Press.
-  Nesterov, Yurii (2004). *Introductory lectures on convex optimization: A basic course*. Vol. 87. Springer Science & Business Media.

Referências V

-  Oja, Erkki (1982). “A simplified neuron model as a principal component analyzer”. Em: *Journal of mathematical biology* 15.3, pp. 267–273.
-  Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.
-  Rasmussen, Carl Edward e Christopher K. I. Williams (2006). *Gaussian processes for machine learning*. The MIT Press.
-  Rosenblatt, Frank (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.”. Em: *Psychological review* 65.6, p. 386.
-  Schölkopf, Bernhard e Alexander J Smola (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.

Referências VI

-  Shalev-Shwartz, Shai e Shai Ben-David (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press.
-  Shannon, C. E. (1988). "Programming a computer for playing chess". Em: *Computer Chess Compendium*. Berlin, Heidelberg: Springer-Verlag, pp. 2–13. ISBN: 0387913319.
-  Shannon, Claude E. (1948). *A Mathematical Theory of Communication*. Vol. 27, pp. 379–423, 623–656.
-  Turing, Alan M. (1969). "Intelligent Machinery". Em: *Machine Intelligence 5*. Ed. por Bernard Meltzer e Donald Michie. Edinburgh University Press, pp. 3–23.
-  Vapnik, Vladimir (1998). *Statistical learning theory*. Wiley.

Referências VII



Vieira, Tim (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025).